UNIAMÉRICA DESCOMPLICA — ENGENHARIA DE SOFTWARE

Disciplina: Containers e Orquestracao de Containers

AGENDA DE ANOTAÇÕES

Tematico: DUNESDAY

Aplicacao full-stack orquestrada com Kubernetes no Minikube

Tecnologias	React 18 + Node.js + Express + MongoDB 7
Orquestracao	Kubernetes (Minikube) + Docker
Replicas	Frontend: 2 | Backend: 2 | MongoDB: 1
Equipe	Heron Felipe / Jihad Ghozayel

Outubro 2026

Sumario

1. Visao Geral do Projeto
2. Arquitetura e Componentes
3. Passo a Passo — Como Executar no Minikube
4. Comandos Uteis
5. Evidencias de Funcionamento
6. Conclusao

1. Visao Geral do Projeto

O DUNESDAY e uma aplicacao full-stack de agenda de anotacoes com tematica inspirada no universo da Latveria (Marvel). O projeto foi desenvolvido originalmente como aplicacao web e adaptado para rodar em um cluster Kubernetes local utilizando Minikube.

O objetivo principal e demonstrar na pratica os conceitos de containerizacao e orquestracao: construir imagens Docker para cada camada da aplicacao, criar manifestos Kubernetes (Deployments, Services, ConfigMaps, Secrets, PVC) e validar comportamentos como self-healing, balanceamento de carga e escalabilidade horizontal.

Funcionalidades:

- Criar, editar e excluir anotacoes (CRUD completo)
- Marcar tarefas como concluidas
- Persistencia dos dados em MongoDB com PersistentVolumeClaim
- Interface tematica com design original (tema Latveria/Doom)
- Health checks nos Pods do backend (liveness/readiness)

2. Arquitetura e Componentes

A aplicacao e composta por tres camadas, cada uma executando em seus proprios containers dentro do cluster Kubernetes:

Componente	Imagem Docker	Replicas	Service	Porta
Frontend	todo-frontend:latest	2	NodePort	80:30080
Backend	todo-backend:latest	2	ClusterIP	8080
MongoDB	todo-mongo:latest	1	ClusterIP	27017

Manifestos Kubernetes:

Arquivo	Tipo	Funcao
mongo-secret.yaml	Secret	Credenciais do MongoDB (base64)
app-configmap.yaml	ConfigMap	URL de conexao do banco
mongo-deployment.yaml	Deployment	Pod do MongoDB com PVC
mongo-service.yaml	Service	Exposicao interna porta 27017
backend-deployment.yaml	Deployment	2 replicas do backend Node.js
backend-service.yaml	Service	Exposicao interna porta 8080
frontend-deployment.yaml	Deployment	2 replicas do frontend React/Nginx
frontend-service.yaml	Service	NodePort :30080 para acesso externo

3. Passo a Passo — Como Executar no Minikube

Pre-requisitos:

- Docker Desktop instalado e rodando
- Minikube instalado (minikube.sigs.k8s.io)
- kubectl instalado (kubernetes.io/docs/tasks/tools)

Passo 1 — Iniciar o Minikube

```
minikube start --driver=docker
minikube status
```

Inicia o cluster local. O status deve mostrar host, kubelet e apiserver como Running.

Passo 2 — Apontar o Docker para o Minikube

```
# Linux/macOS:
eval $(minikube docker-env)

# Windows PowerShell:
& minikube -p minikube docker-env --shell powershell | Invoke-Expression
```

As imagens precisam existir dentro do Minikube. Esse comando redireciona o Docker CLI para o daemon do Minikube. Vale apenas para o terminal atual.

Passo 3 — Construir as imagens

```
docker build -t todo-frontend:latest ./frontend
docker build -t todo-backend:latest ./backend
docker build -t todo-mongo:latest ./banco-de-dados
```

Constroi as tres imagens diretamente no Minikube. Confira com: docker images | grep todo

Passo 4 — Aplicar os manifestos Kubernetes

```
kubectl apply -f k8s/mongo-secret.yaml
kubectl apply -f k8s/app-configmap.yaml
kubectl apply -f k8s/mongo-deployment.yaml
kubectl apply -f k8s/mongo-service.yaml
kubectl apply -f k8s/backend-deployment.yaml
kubectl apply -f k8s/backend-service.yaml
kubectl apply -f k8s/frontend-deployment.yaml
kubectl apply -f k8s/frontend-service.yaml
```

Ordem importa: Secret e ConfigMap primeiro, depois banco, backend e frontend.

Passo 5 — Verificar os recursos

```
kubectl get pods
kubectl get services
kubectl get deployments
```

Todos os 5 Pods devem estar Running com READY 1/1.

Passo 6 — Acessar a aplicacao

```
# Opcao A — tunel do minikube:
minikube service todo-frontend --url

# Opcao B — port-forward:
kubectl port-forward service/todo-frontend 8081:80
```

A opcao A retorna a URL do tunel. A opcao B disponibiliza em localhost:8081. Ambos ficam ativos apenas enquanto o terminal estiver aberto.

4. Comandos Uteis

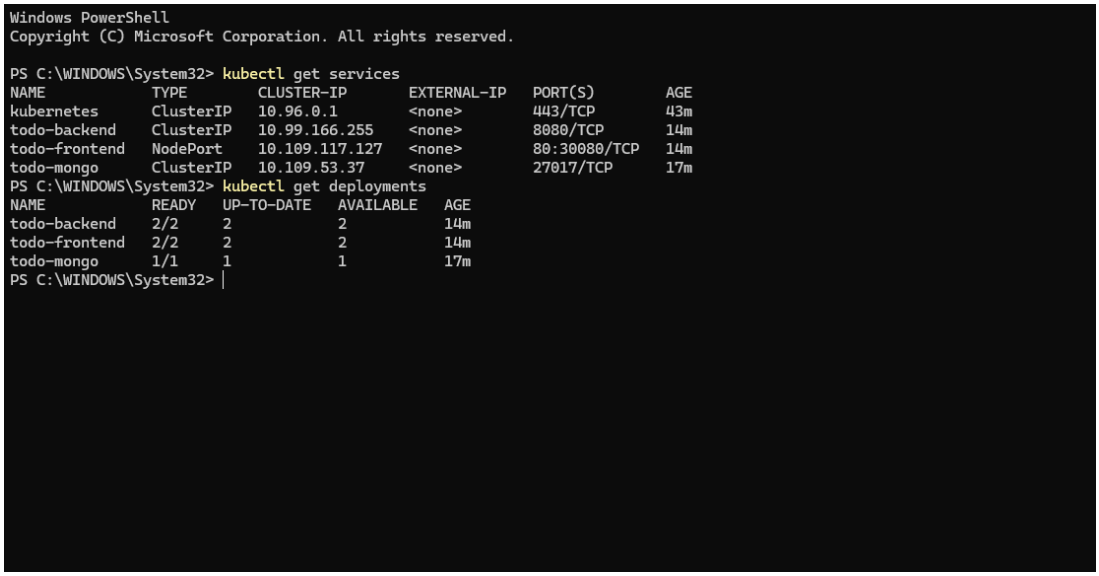
Comando	Funcao
`kubectl get pods`	Lista todos os Pods e status
`kubectl get pods -o wide`	Pods com IP interno e no
`kubectl get services`	Services com tipo e portas
`kubectl get deployments`	Deployments e contagem de replicas
`kubectl logs -l app=todo-backend --tail=10`	Ultimos 10 logs do backend
`kubectl describe pod <nome>`	Detalhes completos de um Pod
`kubectl delete pod <nome>`	Deleta Pod (ReplicaSet recria)
`kubectl rollout restart deploy/<nome>`	Forca redeploy sem downtime
`minikube start`	Inicia ou retoma o cluster
`minikube stop`	Para o cluster (preserva tudo)
`minikube status`	Verifica estado do cluster
`minikube service todo-frontend --url`	Cria tunel para o frontend

5. Evidencias de Funcionamento

As capturas abaixo demonstram o cluster Kubernetes operacional com todos os componentes da aplicacao DUNESDAY em execucao.

5.1 — Services e Deployments

Tres Services (todo-frontend NodePort:30080, todo-backend ClusterIP:8080, todo-mongo ClusterIP:27017) e tres Deployments com todas as replicas disponiveis (2/2, 2/2, 1/1).



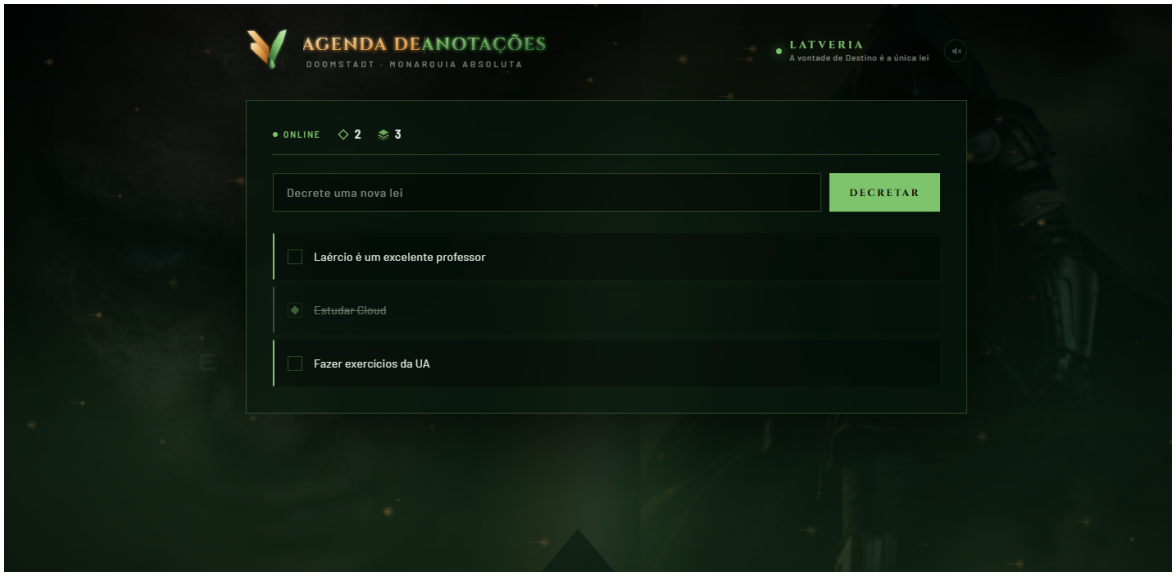
5.2 — Pods com IP interno e no

Cinco Pods rodando no no minikube, cada um com IP proprio na rede 10.244.0.x. Comando: kubectl get pods -o wide.



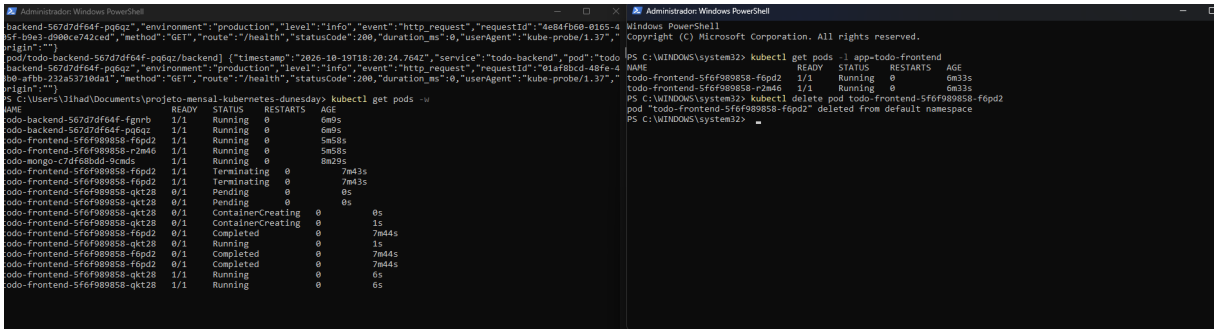
5.3 — Aplicacao em execucao no cluster

Frontend DUNESDAY acessado via tunel Kubernetes — a agenda de anotacoes com tema de Latveria funcionando dentro do cluster.



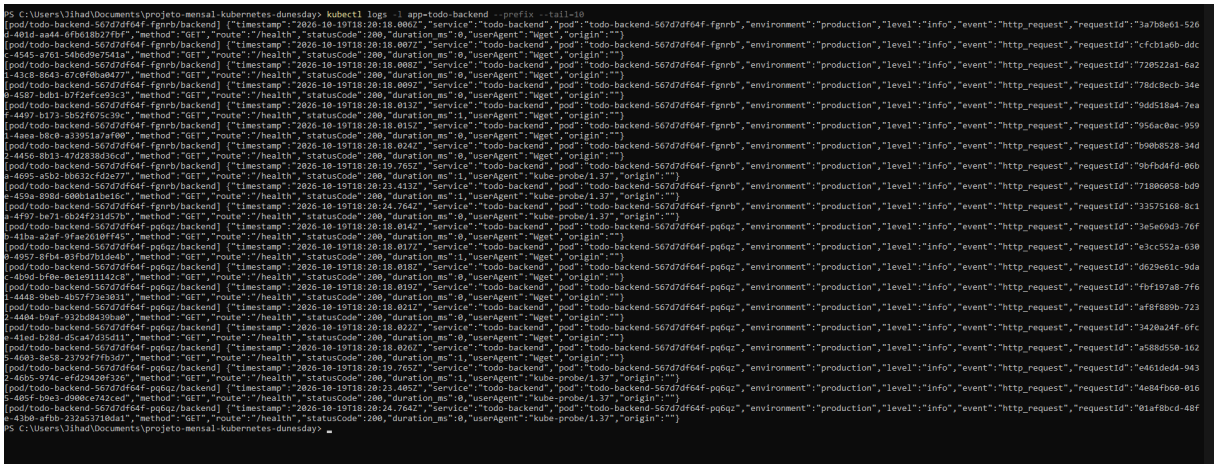
5.4 — Self-healing: Pod deletado volta sozinho

Pod todo-frontend deletado manualmente com kubectl delete pod. O ReplicaSet detectou a ausencia e recriou um novo Pod em segundos, mantendo sempre 2 replicas ativas.



5.5 — Logs do backend com health checks

Logs dos dois Pods backend recebendo requisicoes GET /health do kube-probe. Prova de que o liveness/readiness probe esta configurado.



5.6 — Balanceamento de carga entre replicas

20 requisicoes ao endpoint /api/health: 10 responderam do Pod fgnrb e 10 do Pod pq6qz. O Service distribui a carga igualmente entre as replicas.



6. Conclusao

O projeto DUNESDAY demonstrou com sucesso a implantacao de uma aplicacao full-stack em um cluster Kubernetes local utilizando Minikube. Os principais conceitos praticados foram:

- **Containerizacao:** cada camada da aplicacao (frontend, backend, banco de dados) foi empacotada em sua propria imagem Docker com Dockerfile dedicado.
- **Orquestracao:** os manifestos Kubernetes (Deployments, Services, ConfigMap, Secret, PVC) garantem que a aplicacao rode de forma declarativa e reprodutivel.
- **Self-healing:** ao deletar um Pod manualmente, o ReplicaSet recriou um novo automaticamente em segundos, garantindo a disponibilidade desejada.
- **Balanceamento de carga:** o Service ClusterIP distribuiu as requisicoes igualmente entre as replicas do backend (10/10 em 20 chamadas).
- **Health checks:** os probes de liveness e readiness monitoram continuamente a saude dos containers, reiniciando-os se necessario.
- **Persistencia:** o PersistentVolumeClaim garante que os dados do MongoDB sobrevivam a reinicializacoes do cluster.

O repositorio contem todo o codigo-fonte, Dockerfiles, manifestos YAML e evidencias de funcionamento, permitindo a qualquer pessoa reproduzir a implantacao seguindo o passo a passo documentado.